SAMSUNG

Samsung Innovation Campus

Curso de Inteligencia Artificial

Together for Tomorrow!
Enabling People
Education for Future Generations

Capítulo 3.

Análisis de datos exploratorios: NumPy Arrays para computación numérica optimizada y Pandas

Curso de IA

Descripción del capítulo

Objetivos del capítulo

- ✔ Comprenda el uso preciso de NumPy y sea capaz de procesar datos de manera eficiente.
- ✔ Aprenda los conceptos básicos de NumPy Arrays, indexación y división y las diversas formas de su aplicación.
- ✔ Aprenda a crear y manejar series y objetos de marcos de datos.
- ✔ Conozca los métodos apropiados de ejecución óptima del modelo para el preprocesamiento de datos utilizando Pandas library para explorar y convertir datos.
- ✔ Ser capaz de encontrar el método de análisis adecuado implementando una visualización de datos adecuada a la escala de datos.

Contenido del capítulo

- ✔ Unidad 1. Estructura de datos NumPy array para un rendimiento computacional óptimo
- ✔ Unidad 2. Exploración óptima de datos a través de Pandas
- ✔ Unidad 3. Preprocesamiento de datos de Pandas para una ejecución óptima del modelo
- ✔ Unidad 4. Visualización de datos para varias escalas de datos

Unidad 1.

Estructura de datos NumPy array para un rendimiento computacional óptimo

- 1.1. NumPy Arrays
- 1.2. Conceptos básicos de NumPy array
- 1.3. Operaciones numéricas array
- 1.4. Indexación y corte NumPy
- 1.5. array Transposición e intercambio de ejes

Estructura de datos

- Los conceptos más importantes en programación son los tipos de datos, las estructuras de datos y los algoritmos. Conocer las diferencias claras entre estos tres conceptos es fundamental para manejar fácilmente varios lenguajes de programación y solucionar muchos errores.

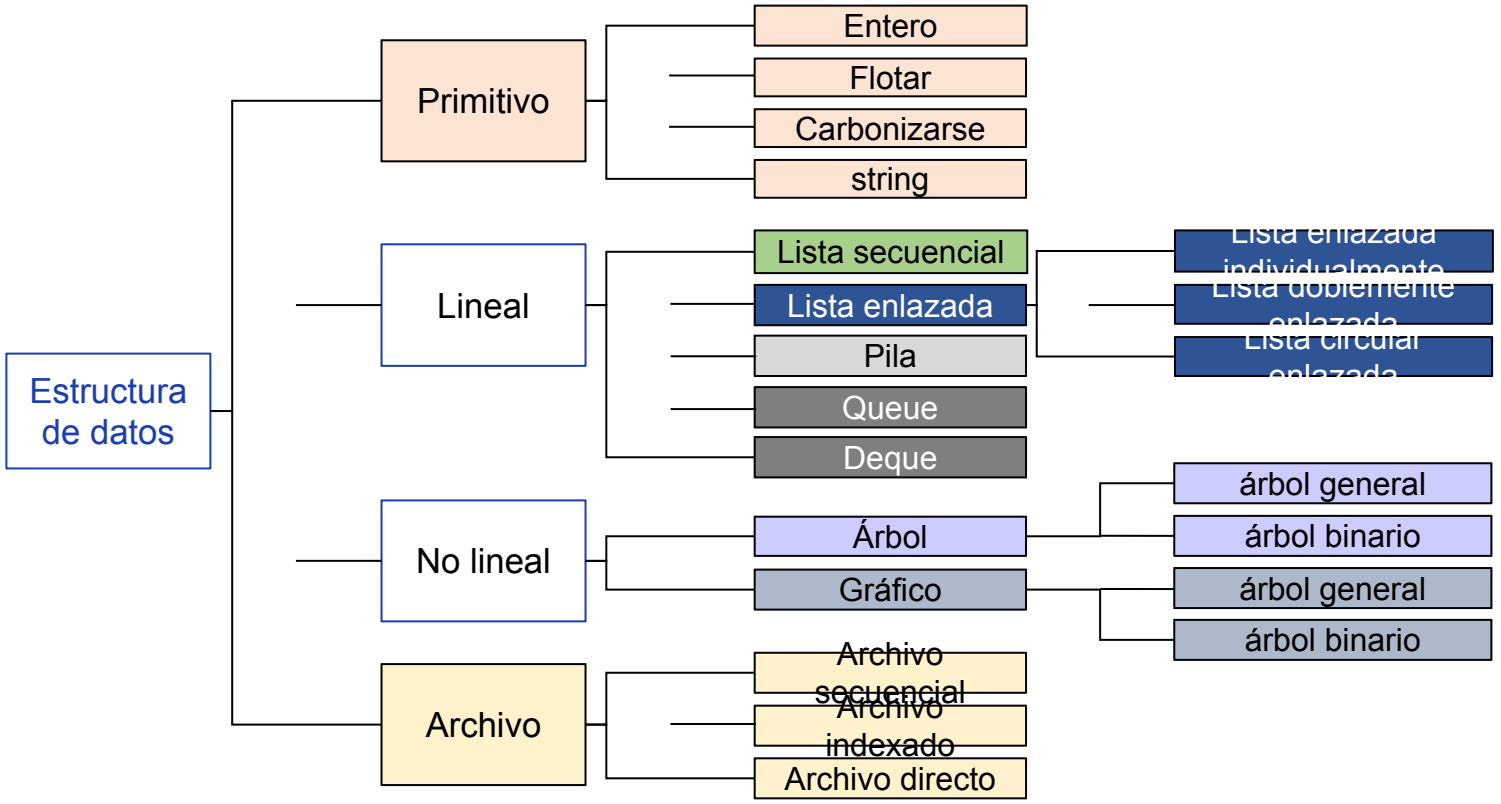
- Tipo de datos
  - Primero, un tipo de datos, en informática y lenguajes de programación, es una clasificación que identifica un tipo de datos como flotantes, enteros, booleanos, caracteres y strings y al mismo tiempo determina el tamaño de los datos. Debe ser utilizado por los tipos de datos aplicables. Los tipos de datos difieren según el lenguaje de programación, pero la mayoría de los conceptos de tipos de datos del lenguaje C se heredan y utilizan y han afectado a otros lenguajes.
  - La memoria es cara, pero cuando se introdujo C, era caro y difícil almacenar una gran cantidad de datos. Como resultado, se diseñó para optimizarse y almacenar el menor espacio necesario, lo que eventualmente resultó en un problema de tamaño de datos.
  - Por ejemplo, en C, los tipos de datos enteros se dividen en char (compatible con números enteros), short, int y long. Cada tamaño de byte difiere de los otros tipos de datos.

- Estructura de datos
  - En segundo lugar, la estructura de datos en informática se refiere a la organización, gestión y almacenamiento de datos que permite un acceso y modificación eficientes. Depende en gran medida de la estructura de datos para encontrar datos de la forma más rápida. En otras palabras, encontrar datos es una cuestión de algoritmos (que se explicará más adelante), por lo que se puede decir que una buena estructura de datos es una condición necesaria y suficiente para un buen algoritmo.
  - Para ser más específico, la estructura de datos se refiere a un grupo de valores de datos, una relación entre datos y una función o comando aplicable a los datos. Las estructuras de datos cuidadosamente seleccionadas permiten utilizar algoritmos más eficientes.
  - Una estructura de datos diseñada eficazmente permite realizar operaciones con recursos mínimos, como tiempo de ejecución o capacidad de memoria.
  - Existen varios tipos de estructuras de datos, cada una de las cuales se adapta a cada operación y propósito.
  - Al diseñar varios programas, la prioridad debe ser considerar y seleccionar la estructura de datos más adecuada. Esto se debe a que cuando se fabrica un sistema a gran escala, la dificultad de implementación y el rendimiento del producto final dependen en gran medida de la estructura de datos.
  - Una vez seleccionada la estructura de datos, queda relativamente claro qué algoritmo se debe aplicar. Hay ocasiones en las que este orden se invierte y es cuando la operación de destino requiere necesariamente un algoritmo particular, y el algoritmo dado produce el mejor rendimiento con la estructura de datos particular. En cualquier caso, es fundamental seleccionar una estructura de datos adecuada.

- Algoritmo
  - En tercer lugar, un algoritmo es una formulación de un conjunto de procedimientos o métodos para resolver cualquier problema solucionable y se refiere a un procedimiento paso a paso para ejecutar un cálculo.
  - Los algoritmos son los más importantes en los campos de Machine Learning y Deep Learning, áreas que cubriremos en el futuro porque requiere trabajar con muchos datos.
  - Al final, como el rendimiento del algoritmo está directamente relacionado con el rendimiento de la estructura de datos, se puede decir que lo más importante es saber con precisión dónde utilizar una determinada estructura de datos.

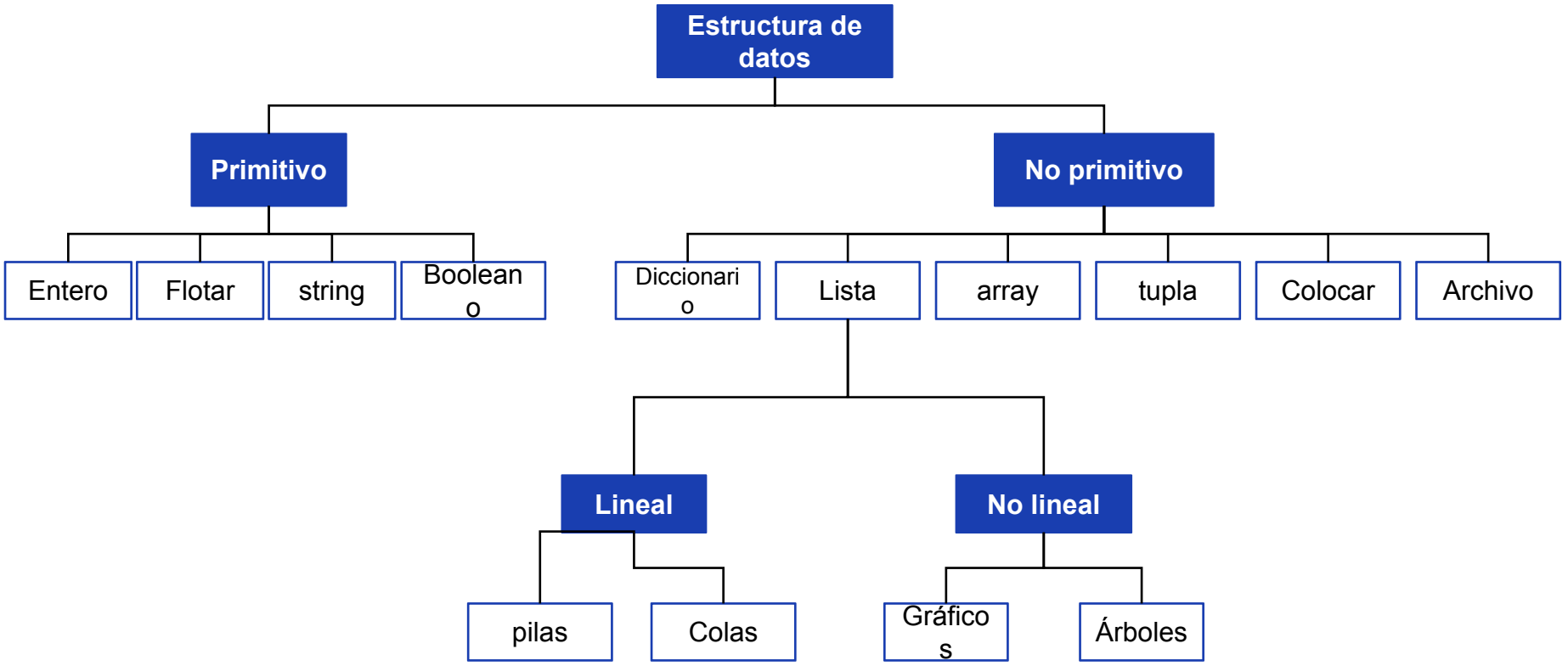
Estructuras de datos en Python

- Primero, los siguientes son los tipos de estructuras de datos utilizados en informática:



- Tipos de estructuras de datos (1/2)
  - Sin embargo, aquí sólo nos ocuparemos de la estructura de datos del lenguaje Python. En general, se puede dividir en gran medida en una estructura de datos primitiva, una estructura para tipos básicos y una estructura de datos no primitiva, una estructura para almacenar de manera efectiva múltiples datos con tipos de datos básicos.
  - La estructura de datos no primitiva más común y convenientemente utilizada en Python es la lista. Esto se debe a que el tamaño de los datos no es fijo al almacenar varios tipos de datos diferentes (entero, caracter, etc.)
  - Sin embargo, la ventaja se ha vuelto inapropiada en los campos Machine Learning y Deep Learning, donde se deben procesar más datos de forma rápida y exclusivamente en números.
  - Esto se debe a que las computadoras deben tomar números decimales comprendidos por los humanos y convertirlos en formas binarias para realizar operaciones rápidas y usar los mismos datos numéricos para acceder directamente y calcular cada elemento sin repetición.
  - Al final, las computadoras sólo pueden realizar operaciones cuando las letras/caracteres se convierten en números.

- Tipos de estructuras de datos (2/2)
  - Para compensar tales deficiencias está la estructura de datos array. Python, sin embargo, no es compatible directamente con array; más bien, la estructura de datos array se puede utilizar a través de NumPy library.

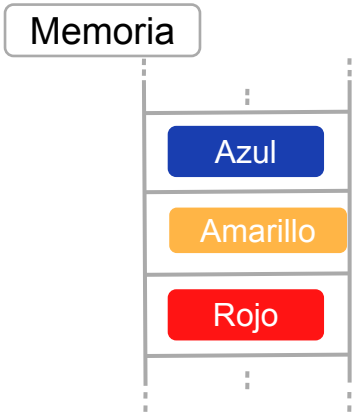


- Método de almacenamiento de memoria por estructura de datos
  - Para comprender por qué las estructuras de datos array permiten operaciones tan rápidas, primero debemos saber cómo se almacenan en la memoria para cada estructura de datos.
  - Normalmente, en la mayoría de los idiomas, se hace referencia a array como un grupo de datos creado al enumerar datos del mismo tipo y almacenarlos de forma contigua en la memoria.
  - Cada valor que contiene se denomina elemento de array. Estos elementos utilizan el número llamado índice, que siempre comienza en cero, para simplemente distinguir los elementos de un array. La mayoría de los tipos de datos se pueden configurar en una disposición y constan de una disposición unidimensional, una disposición bidimensional y una disposición tridimensional, según el tipo de configuración.
  - Anteriormente se explicó que la estructura de datos era un concepto que se centraba principalmente en almacenar datos de forma eficaz. Descubrir cómo se almacenan los datos en el array y la lista es la forma precisa de comprender los pros y los contras del array y conocer el motivo de su uso.

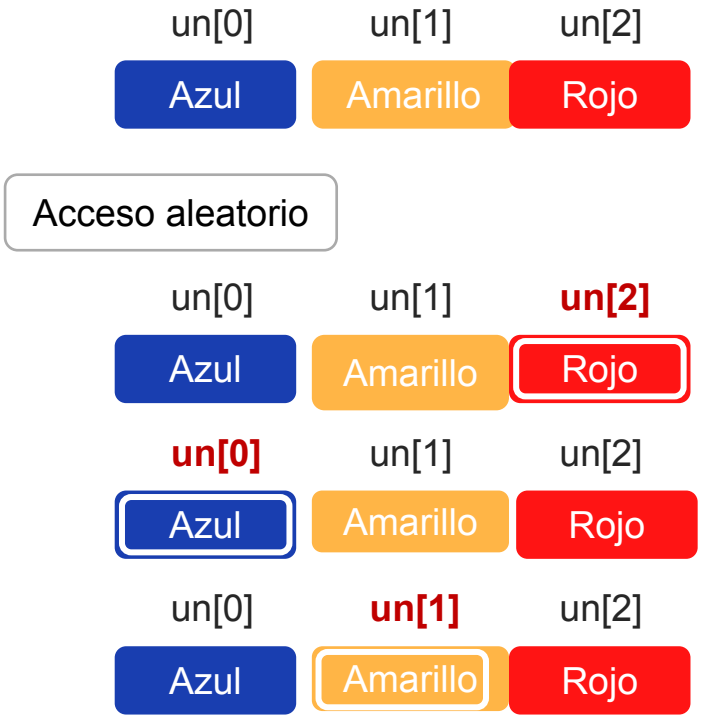
- Almacenamiento y Acceso de Datos en un array (1/2)
  - El siguiente es el proceso de acceso a datos existentes al almacenarlos en un array y agregar y eliminar datos nuevos. Debes compararlo con la lista que sigue.
  - Supongamos que tres datos con strings que representan los siguientes colores se almacenan en un array.



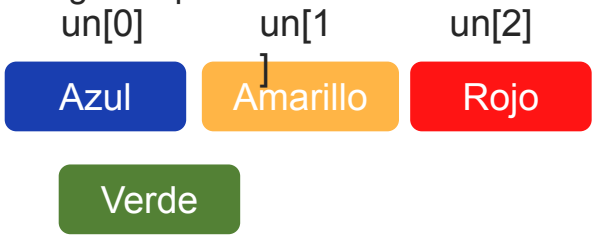
  - Se puede acceder a cada elemento a través de cada índice. Un índice es un número que representa un pedido. Los datos se almacenan secuencialmente en una ubicación contigua de la memoria, como se muestra en la siguiente figura.



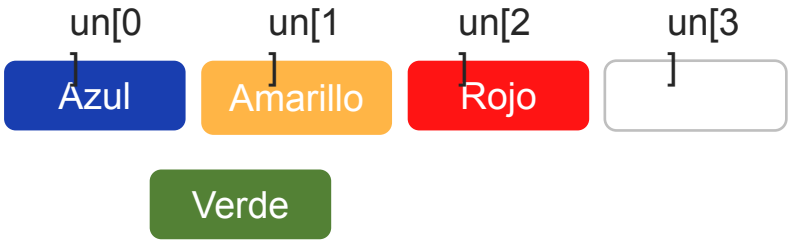
- Almacenamiento y Acceso de Datos en un array (2/2)
  - Dado que los datos se almacenan en una ubicación contigua, se puede acceder a la dirección de la memoria con un índice y los datos se pueden seleccionar aleatoriamente para acceder a la ubicación deseada de los datos. Aquí hay una imagen donde los datos se acercan al rojo en la tercera habitación (expresaremos el concepto de una variable que almacena principalmente un valor como una habitación) y se acercan a la habitación azul. El acceso aleatorio es posible a través del índice.



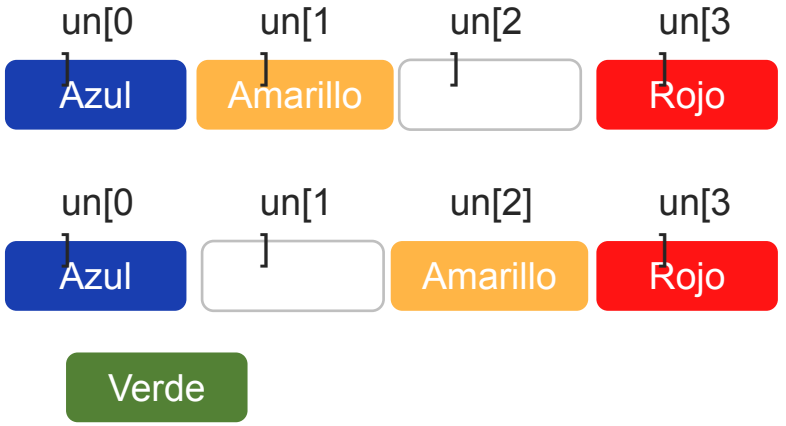
- Agregar y eliminar datos del array (1/4)
  - Otra característica de array es que agregar o eliminar datos en una ubicación específica requiere más cálculo y espacio en comparación con la lista. El cálculo aquí es que la CPU necesita más tiempo para calcular y también se necesita más espacio en la memoria.
  - Considere agregar el valor "Verde" a la segunda posición.



  - En primer lugar, debemos asegurar espacio adicional al final del arreglo.



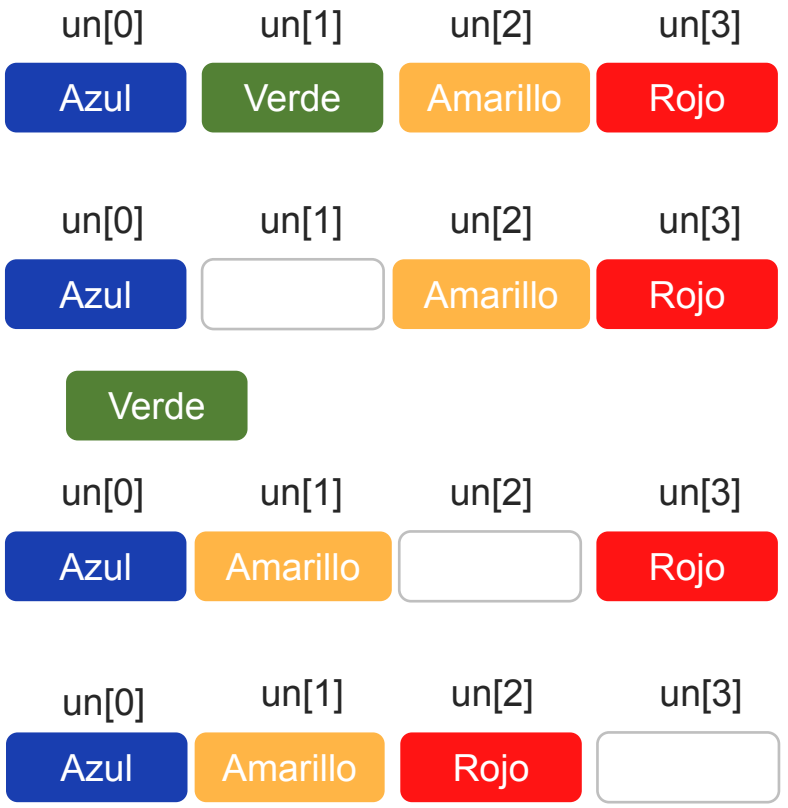
- Agregar y eliminar datos del array (2/4)
  - Para agregar datos al segundo espacio, los datos detrás del segundo espacio deben moverse hacia la derecha uno por uno.



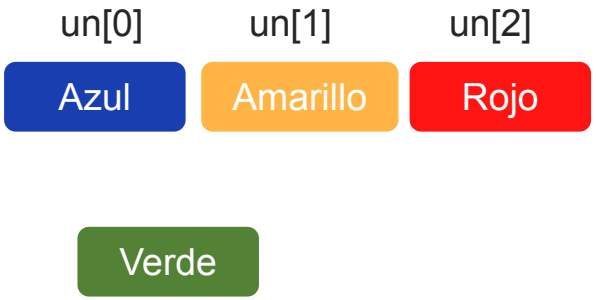
  - Luego, los datos "verdes" se agregan al espacio vacío.



- Agregar y eliminar datos del array (3/4)
  - Por el contrario, al eliminar el segundo elemento, el valor "Verde", primero elimine el elemento y luego mueva el valor hacia la izquierda uno por uno para que no quede ningún espacio vacío.



- Agregar y eliminar datos del array (4/4)
  - Se completa eliminando el último espacio restante.

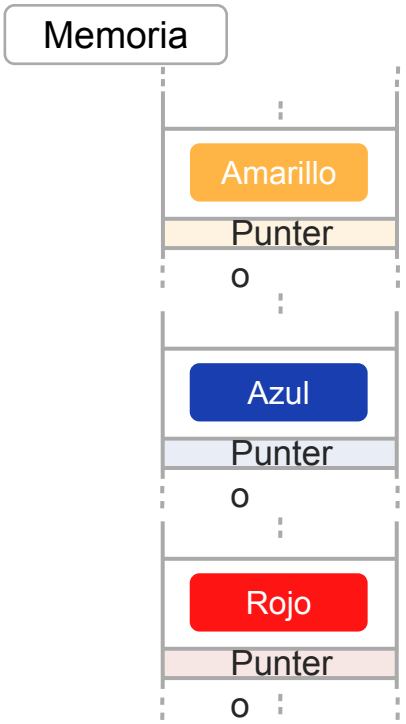


- Guardar y acceder a datos de la lista (1/4)
  - El siguiente es un método para almacenar datos en la estructura de datos de la lista.

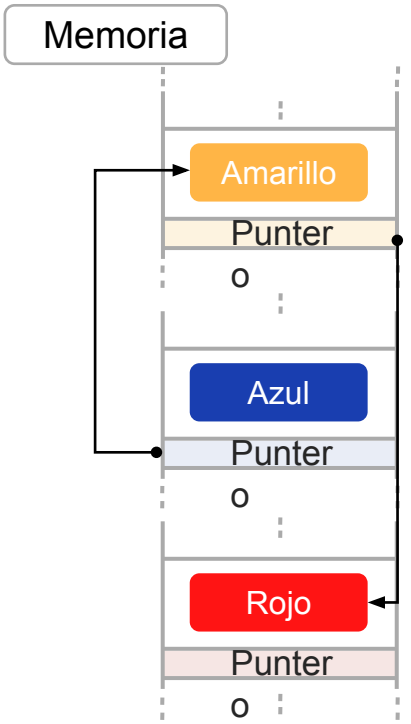


  - Para comprender las listas, primero debe comprender el concepto de puntero. En términos simples, un puntero es un valor de dirección que apunta a un valor determinado.
  - Como se muestra en la figura anterior, la habitación azul (el concepto de una variable que representa un valor se expresará como una habitación) apunta a la habitación amarilla. Digamos que la habitación azul tiene la dirección de la ubicación de memoria de la habitación amarilla. Entonces cada habitación puede apuntar a la siguiente habitación.

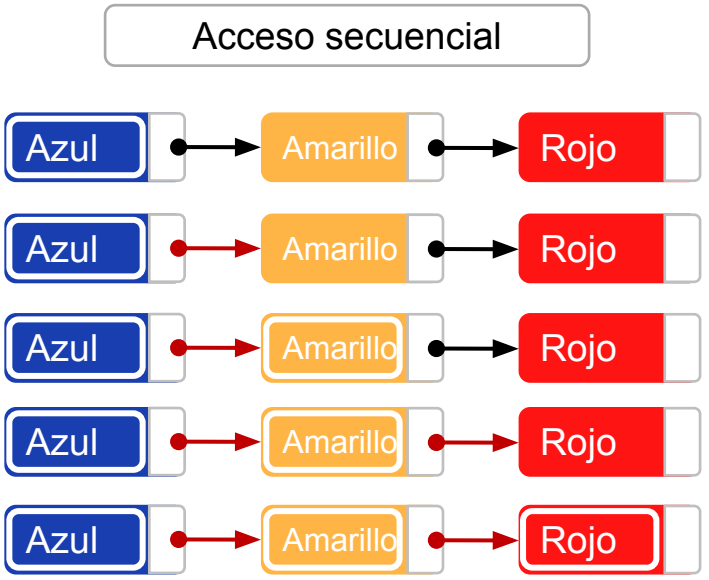
- Guardar y acceder a datos de la lista (2/4)
  - Las listas no se almacenan secuencialmente sino en ubicaciones separadas.



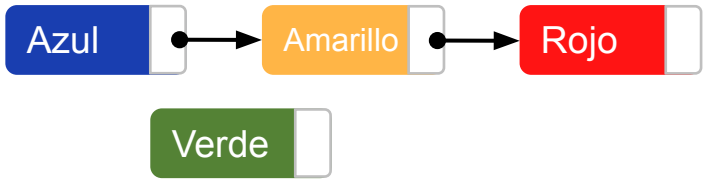
- Guardar y acceder a datos de la lista (3/4)
  - Dado que el puntero del valor "Azul" se refiere a la dirección del valor "Amarillo" y el puntero del valor amarillo se refiere al valor "Rojo", se puede mantener el orden.

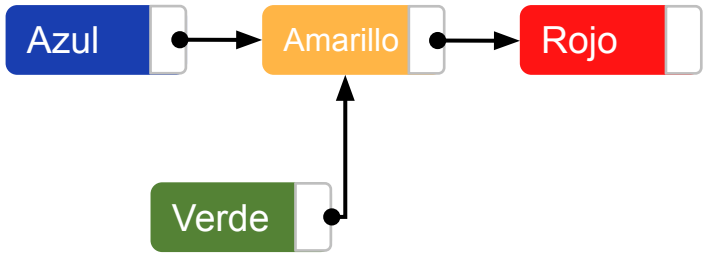


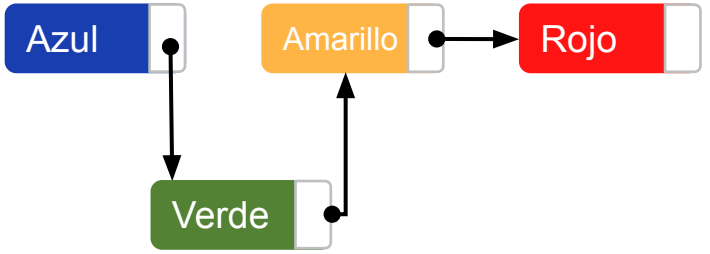
- Guardar y acceder a datos de la lista (4/4)
  - Dado que los datos se almacenan en direcciones no contiguas, se accede a los enfoques secuenciales a través del puntero que los precede.



- Agregar y eliminar datos de la lista
  - Ahora, veamos el caso de agregar los datos "verdes".







  - Como se muestra en la imagen de arriba, usando cada puntero, el azul apunta al verde y el verde apunta al amarillo para agregar el verde.

¿Qué es NumPy?

- NumPy significa Numerical Python, que es un paquete básico para ciencia de datos.
  - Es un Python library que proporciona objetos array multidimensionales, varios objetos derivados (matrices, etc.) y una variedad de rutinas para una operación rápida en Arrays.
  - Admite transformadas discretas de Fourier, álgebra lineal básica, operaciones estadísticas básicas, simulaciones aleatorias, etc.
  - El objeto ndarray es el núcleo del paquete NumPy. Procesa Arrays n-dimensional de tipos de datos homogéneos.

- Distinga las diferencias entre NumPy Arrays y las secuencias estándar de Python (listas, tuplas, diccionarios, conjuntos).
  - NumPy Arrays tiene un tamaño fijo cuando se genera a diferencia de las listas de Python (que pueden crecer dinámicamente).
  - Un cambio en el tamaño del ndarray crea un nuevo array y elimina el original.
  - Los elementos de NumPy array son todos tipos de datos homogéneos; por tanto, mantienen el mismo tamaño en la memoria.
  - Sin embargo, las listas permiten Arrays de varios tamaños.
  - NumPy Arrays calcula fácilmente operaciones matemáticas avanzadas y de otro tipo en grandes cantidades de datos.
  - Normalmente, estas operaciones se realizan de forma más eficiente y con menos código que cuando se realizan con las secuencias integradas de Python.
  - Más paquetes científicos y matemáticos basados en Python utilizan NumPy Arrays. Por lo general, estos admiten la entrada de secuencia de Python, pero convierten dicha entrada a NumPy Arrays antes de procesarla y, a menudo, generan NumPy Arrays.
  - En otras palabras, no es suficiente confiar únicamente en las secuencias integradas de Python para utilizar eficientemente el software científico/matemático actual basado en Python. También es necesario tener conocimientos de NumPy array para aumentar la eficiencia.

- ¿Por qué utilizar NumPy Arrays? (1/3)
  - Anteriormente, aprendimos que Python no admite directamente Arrays pero permite la disposición de la estructura de datos a través de NumPy library.
  - Los conceptos de organización aprendidos en los lenguajes de programación estándar no son muy diferentes en Python.
  - En programación, disminuir los bucles es el método para mejorar el rendimiento.
  - Los bucles en cálculos a gran escala requieren un cálculo por repetición, lo que provoca un rendimiento deficiente.
  - NumPy Arrays permite una amplia variedad de operaciones de procesamiento de datos a través de operaciones concisas array en lugar de bucles.
  - El uso de la informática array para eliminar bucles explícitamente se denomina vectorización. Las operaciones matemáticas para Arrays vectorizado suelen ser dos o tres, si no diez o incluso cien veces más rápidas que las operaciones puras de Python. La transmisión, otro método que aprenderemos más adelante, es una operación vectorial muy poderosa.

- ¿Por qué utilizar NumPy Arrays? (2/3)
  - Dado que la operación NumPy array utiliza una iteración interna implementada en C, es más rápida que la iteración de Python y realiza una operación algebraica lineal utilizando una operación vectorizada.
  - Una operación de vectorización es una de las transformaciones lineales que transforman una matriz en un vector vertical, como se muestra a continuación.
  - Dado que funciona en todo el vector, se puede utilizar en lugar de los bucles for y while.

matriz 2x2 $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$ a través de la vectorización se convierte en $vec(A) = \begin{bmatrix} a \\ c \\ b \\ d \end{bmatrix}$

  - En scikit-learn, la disposición NumPy es la estructura de datos básica. En otras palabras, la disposición NumPy debe usarse como entrada/salida estándar para Machine Learning.

- ¿Por qué utilizar NumPy Arrays? (3/3)
  - En scikit-learn, la disposición NumPy es la estructura de datos básica. En otras palabras, la disposición NumPy debe usarse como entrada/salida estándar para Machine Learning.
  - Scikit-Learn es un Python Machine Learning library.
  - Dado que scikit-learn recibe datos en forma de NumPy array como entrada, todos los datos que se utilizarán en el futuro deben convertirse en NumPy array.

Unidad 1.

Estructura de datos NumPy array para un rendimiento computacional óptimo

- 1.1. NumPy Arrays
- 1.2. Conceptos básicos de NumPy array
- 1.3. Operaciones numéricas array
- 1.4. Indexación y corte NumPy
- 1.5. array Transposición e intercambio de ejes

Cómo importar NumPy

- NumPy library Importar
  - Para una mejor legibilidad del código, abrevie NumPy como np. Esta es una regla adoptada por todos aquellos que trabajan con códigos para que pueda entenderse fácilmente.
  - En Python, los objetos generalmente tienen propiedades y métodos. La propiedad es otro objeto de Python almacenado dentro del objeto y el método se refiere a una función que permite el acceso a los datos internos del objeto. Se puede abordar en forma de np.attribute_name.

```
[1]: import numpy as np

[2]: np.__version__

[2]: '1.24.3'
```

Línea 1

- as se utiliza para crear un alias, lo que significa que el nombre del módulo se abrevia.

Línea 2

- Para ver la versión de NumPy, use la propiedad incorporada __version__.

Conceptos básicos de NumPy array

- NumPy ndarray
  - Un objeto array de n dimensiones se llama ndarray. Se puede utilizar para procesar y almacenar grandes conjuntos de datos.
  - Los arreglos rápidos y flexibles utilizan una gramática similar a la utilizada para operaciones entre elementos escalares. Utilizan operaciones matemáticas para todo el bloque de datos.
- En primer lugar, se puede convertir un arreglo en una secuencia (lista, tupla, array, conjunto).

```
[3]: np.array([1,3,4,5,9])

[3]: array([1, 3, 4, 5, 9])

[4]: arr1=np.array([1,3,5,7,9])

[5]: type(arr1)

[5]: numpy.ndarray
```

- Aquí hay un arreglo más.

```
[6]: arr2=np.array([1,3,5,7,9])
```

- Verifique la dirección de cada array usando una función que representa la identificación de la dirección.

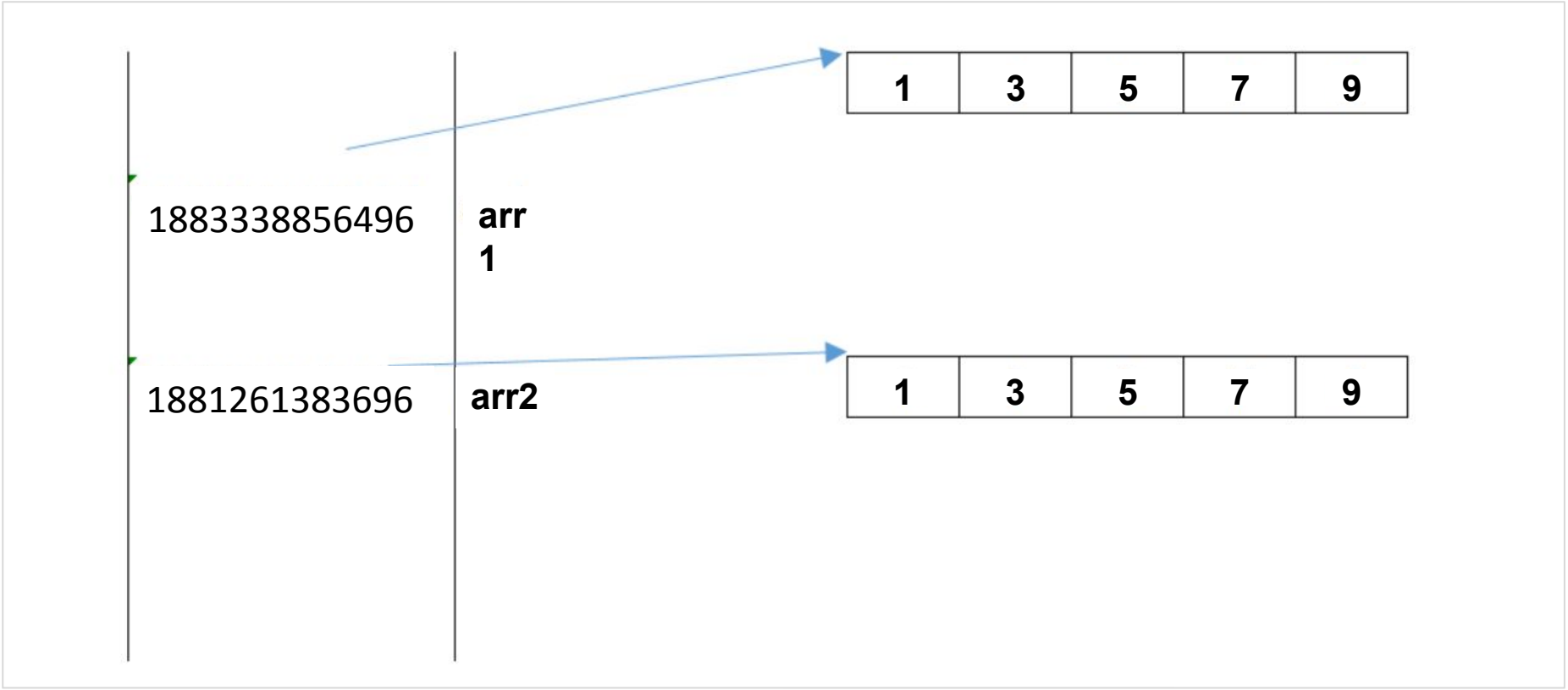
```
[7]: id(arr1)

[7]: 1883338856496

[8]: id(arr2)

[8]: 1881261383696
```

- El valor real dentro es el mismo, pero se confirma que la dirección es diferente porque es su propio objeto.



- En Python, el símbolo = es un operador de asignación que asigna valores de dirección.

- Asignemos la dirección arr1 a la variable arr3.

```
[9]: arr3=arr1

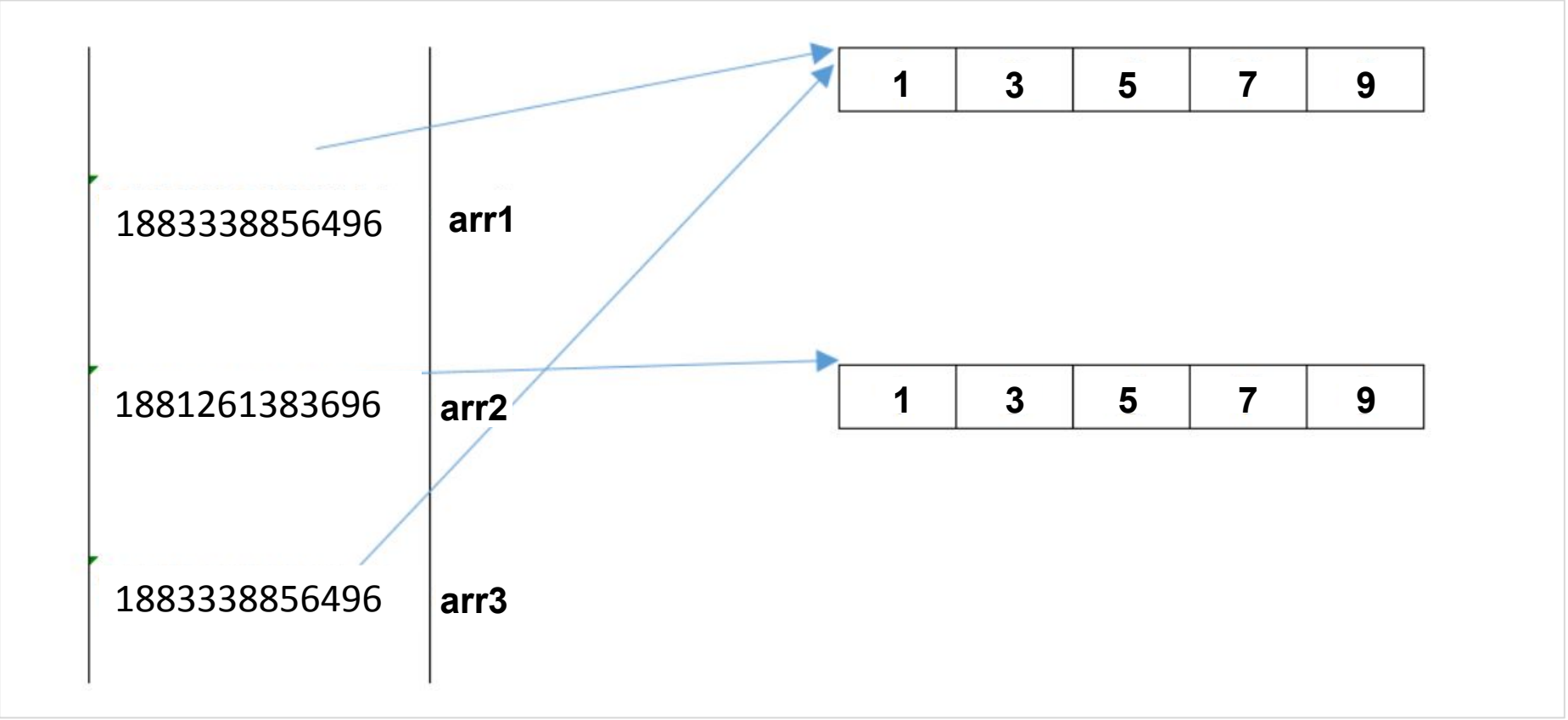
[10]: print(id(arr1))
      print(id(arr3))

      1883338856496
      1883338856496
```

Línea 9

- Después de asignar los valores de dirección, podemos ver que los valores de dirección son los mismos.

- Podemos ver que los valores de dirección son los mismos.



- Si desea copiar solo el valor, puede usar np.copy(). Consulte ayuda (np.copy) para obtener más detalles.

```
[11]: arr4=np.copy(arr1)

[12]: arr4

[12]: array([1, 3, 5, 7, 9])
```

- Luego verifiquemos el valor de la dirección.

```
[13]: id(arr4)

[13]: 1881261383888
```

Línea 13

- Como solo se copió el valor, puede ver que el valor de la dirección es diferente.

- Creando Arrays con tuplas

```
[14]: type((1,3,5,7,9))

[14]: tuple

[15]: np.array((1,3,5,7,9))

[15]: array([1, 3, 5, 7, 9])
```

- Crear Arrays con diccionarios

```
[16]: type({'one':1, 'two':2, 'three':2})

[16]: dict

[17]: np.array({'one':1, 'two':2, 'three':2})

[17]: array({'one': 1, 'two': 2, 'three': 2}, dtype=object)
```

- Creando Arrays con sets

```
[18]: {1,2,2,2,2,3,3,3,4}

[18]: {1, 2, 3, 4}

[19]: type({1,2,2,2,2,3,3,3,4})

[19]: set

[20]: np.array({1,2,2,2,2,3,3,3,4})

[20]: array({1, 2, 3, 4}, dtype=object)
```

  - Normalmente, las listas se utilizan habitualmente para crear Arrays.

- Creando Arrays con arange
  - Este es el mismo concepto que el rango en Python estándar.

```
[21]: [i for i in range(10)]

[21]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

[22]: [i for i in np.arange(10)]

[22]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

[23]: np.arange(10)

[23]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

[24]: np.arange(1,10)

[24]: array([1, 2, 3, 4, 5, 6, 7, 8, 9])

[25]: np.arange(1,10,2)

[25]: array([1, 3, 5, 7, 9])
```

  - Para NumPy Arrays, utilice np.arange.

- Utilice el método len() y la propiedad size para el tamaño de array.

```
[26]: len10=np.arange(10)

[27]: len(len10)

[27]: 10

[28]: len10.size

[28]: 10
```

- NumPy array trata cada elemento convirtiéndolo al mismo tipo de datos. (1/3)
  - Lo importante es que el NumPy Arrays procesa el mismo tipo de forma rápida y efectiva, como una de sus características.
  - Sin embargo, aunque Arrays se puede fabricar con diferentes tipos de Arrays, cada tipo se convierte en el mismo tipo.

```
[29]: np.array([111,2.3,True])

[29]: array([111. ,   2.3,   1. ])
```

Línea 29

- Se pueden crear diferentes tipos de Arrays, como números enteros, flotantes y booleanos.

- NumPy array trata cada elemento convirtiéndolo al mismo tipo de datos. (2/3)
  - Debido a la. en el tipo entero, array se convierte en flotante cuando los tipos entero, flotante y booleano están juntos.

```
[30]: arr5=np.array([111,2.3,True])

[31]: type(arr5[0])

[31]: numpy.float64

[32]: arr6=np.array([111,2.3,'hi'])
```

Línea 31

- El tipo de datos de NumPy se devuelve en formato NumPy.datatype.

Línea 32

- Se crean diferentes tipos de Arrays, como números enteros, flotantes y strings.

- NumPy array trata cada elemento convirtiéndolo al mismo tipo de datos. (3/3)

```
[33]: arr6[0]

[33]: '111'

[34]: type(arr6[0])

[34]: numpy.str_
```

Línea 34

- Debido al '111' en el tipo entero, array se convierte en string cuando los tipos entero, flotante y string están juntos.

- Los tipos de datos de NumPy Arrays son los siguientes:

Tipo de datos	Explicación
int8, int16, int32, int64, int_ uint8, uint16, uint32, uint64	Entero
float16, float32, float64, float128, float_	punto flotante
bool_	Booleano
cadena_, unicode_	string

  - El 8 en int8 son 8 bits, que es el rango de valores que puede representar este valor.

https://numpy.org/devdocs/reference/Arrays.dtypes.html

- Explicación de 8 bits.

```
[35]: 2**8

[35]: 256
```

- Dado que el bit representa dos valores, 8 bits pueden convertirse en 2 elevado a 8, lo que representa hasta 256 números enteros. En números enteros, este valor varía de -128 a 127. El rango es hasta 127, y no 128, incluido cero y un valor está reservado para el lado negativo.
- Si solo se utilizan valores positivos, el espacio en el rango de números negativos debe cruzarse con números positivos. En este momento se utiliza u, la primera letra del sin firmar (que significa positivo).
- Por lo tanto, uint8 oscila entre 0 y 255.

- Tipos de datos de NumPy Arrays

```
[36]: np.array(['apple', 'banana', 'strawberry'])

[36]: array(['apple', 'banana', 'strawberry'], dtype='<U10')

[37]: arr7=np.array(['apple', 'banana', 'strawberry'])

[38]: type(arr7[0])

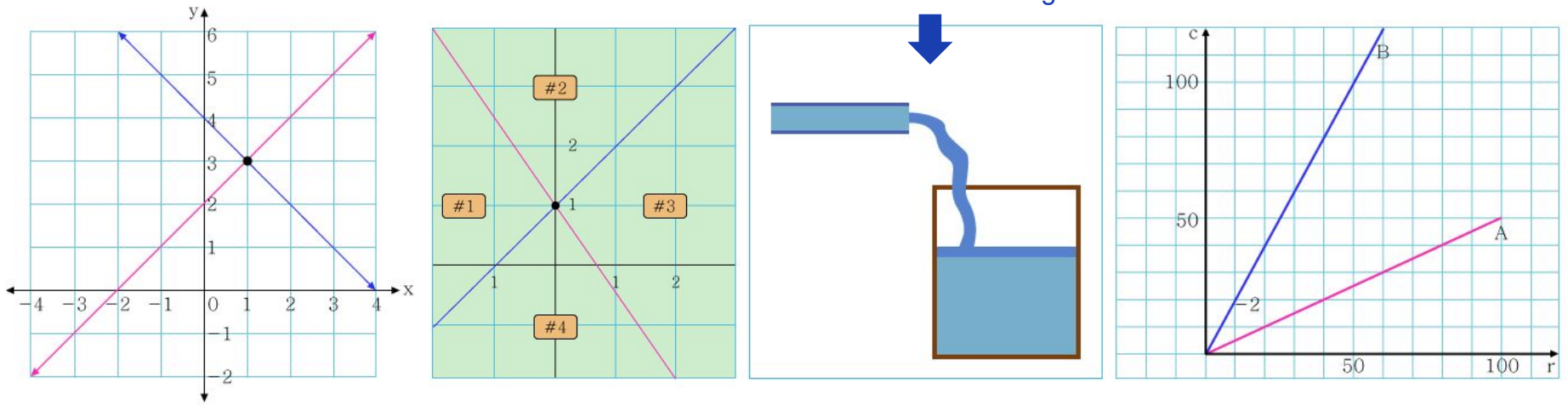
[38]: numpy.str_
```

Línea 36

- El U10, aquí, representa Unicode, and el símbolo < representa 10 bytes or menos.

- Creación de Arrays con la función linspace (1/2): consulte la ayuda (np.linspace) para obtener más detalles.
  - Linear means that it can be expressed in the form of linear bonds with respect to the elements of set A.
  - That is, where the elements of set A $x_1, x_2, x_3, \ldots, x_n$ are multiplied and added by constants $a_1, a_2, a_3, \ldots$, forming $a_1x_1 + a_2x_2 + a_3x_3 + \cdots + a_nx_n$, which belongs to this set A. This type of expression $x_1, x_2, x_3, \ldots, x_n$ is called a linear combination.
  - In this way, representing the heat of the basic element in the form of a linear bond is called linear. Linear combination is the corresponding coefficients and variables, and the solution can be obtained through this equation.
  - The figure below shows the concept of linearity in mathematics and daily life.

Relación lineal entre el tiempo y la altura de la botella de agua.



- Creando Arrays con la función linspace (2/2)

```
[39]: np.linspace(1,10,5)

[39]: array([ 1.  ,  3.25,  5.5 ,  7.75, 10.  ])

[40]: np.linspace(-10,10,20)

[40]: array([-10.        ,  -8.94736842,  -7.89473684,  -6.84210526,
             -5.78947368,  -4.73684211,  -3.68421053,  -2.63157895,
             -1.57894737,  -0.52631579,   0.52631579,   1.57894737,
              2.63157895,   3.68421053,   4.73684211,   5.78947368,
              6.84210526,   7.89473684,   8.94736842,  10.        ])
```

Línea 39

- Cinco números en el espacio lineal del 1 al 10.

Línea 40

- 20 números en el espacio lineal del -10 al 10.

- Creando Arrays con np.zeros() y np.ones()

```
[41]: np.zeros(5)

[41]: array([0., 0., 0., 0., 0.])

[42]: np.ones(5)

[42]: array([1., 1., 1., 1., 1.])
```

Línea 41

- Tenga cuidado de no saltarse la "s".

Multidimensional Arrays

- Un array multidimensional se refiere a un array de dos o más dimensiones. Se puede hacer en forma de lista en la lista.

```
[43]: li1=[[1,2,3],[4,5,6],[7,8,9]]

[44]: li1[0][0]

[44]: 1

[45]: np.array(li1)

[45]: array([[1, 2, 3],
             [4, 5, 6],
             [7, 8, 9]])

[46]: arr2d=np.array(li1)
```

Línea 44

- El primer elemento de esta lista también es una lista, por lo que el primer elemento de esa lista es 1.

Línea 45

- array bidimensional

- Comprueba las dimensiones del array con ndim.

```
[47]: arr2d.ndim

[47]: 2
```

- Luego, verifiquemos el array unidimensional que aprendimos antes.

```
[48]: arr1d=np.array([1,3,5,7,9])

[49]: arr1d.ndim

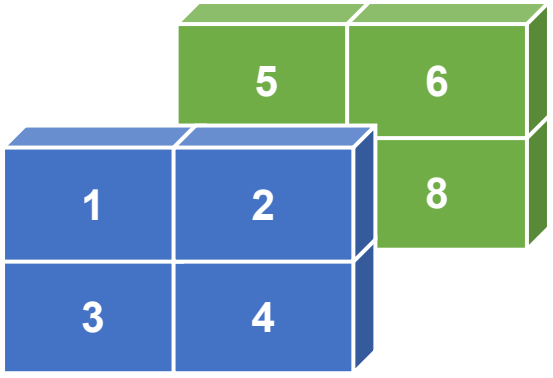
[49]: 1
```

- También realizaremos un array tridimensional.

```
[50]: np.array([[[1,2],[3,4]], [[5,6],[7,8]]])

[50]: array([[[1, 2],
              [3, 4]],

             [[5, 6],
              [7, 8]]])
```



- Tridimensional array

```
[51]: arr3d=np.array([[[1,2],[3,4]], [[5,6],[7,8]]])

[52]: arr3d.ndim

[52]: 3

[53]: arr3d[0][0][0]

[53]: 1
```

Línea 51

- Una superficie es bidimensional y se puede apilar. Es fácil de entender si piensas en la superficie como si fuera una hoja de papel.

Línea 53

- Al extraer un valor tridimensional, es más fácil pensar en ello como un método de aproximación paso a paso.

- Creando formas bidimensionales usando np.zeros()

```
[54]: np.zeros((2,3))

[54]: array([[0., 0., 0.],
             [0., 0., 0.]])
```

Línea 54

- Desígnelo como dos filas y tres columnas en forma de tupla.

- Cuando se crea, el elemento que contiene normalmente es un flotante.

```
[55]: arr2dZero=np.zeros((2,3))

[56]: type(arr2dZero[0][0])

[56]: numpy.float64
```

- Si lo crea un tipo diferente, designe el nombre del parámetro usando dtype.

```
[57]: arr2dZero2=np.zeros((2,3), dtype='int64')

[58]: arr2dZero2

[58]: array([[0, 0, 0],
             [0, 0, 0]], dtype=int64)

[59]: type(arr2dZero2[0][0])

[59]: numpy.int64
```

- Utilice el método astype() si desea cambiar el tipo a flotante.

```
[60]: arr2dZero2.astype('float32')

[60]: array([[0., 0., 0.],
             [0., 0., 0.]], dtype=float32)

[61]: arr2dZero2=arr2dZero2.astype('float32')

[62]: arr2dZero2

[62]: array([[0., 0., 0.],
             [0., 0., 0.]], dtype=float32)
```

Propiedades del NumPy array

- Explicación de cada propiedad
  - Tenga en cuenta que, dado que no es un método, no se debe adjuntar ().

```
[63]: a=np.array([[1,2,3],[4,5,6],[7,8,9]])

[64]: a.size

[64]: 9

[65]: a.shape

[65]: (3, 3)

[66]: a.ndim

[66]: 2
```

Línea 64

- Tamaño

Línea 65

- Convertir la fila y las columnas en una tupla

- Cabe señalar que para Arrays unidimensional, la forma devuelve una tupla con el número de elementos.

```
[67]: a1=np.array([2,5,1,3])

[68]: a1.shape

[68]: (4,)
```

remodelar

- Cómo remodelar

```
[69]: a2=np.arange(15)

[70]: a2

[70]: array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14])

[71]: a2.reshape((3,5))

[71]: array([[ 0,  1,  2,  3,  4],
             [ 5,  6,  7,  8,  9],
             [10, 11, 12, 13, 14]])
```

Línea 71

- Asegúrese de ingresar la forma como una tupla.

- Comprobando la forma del objeto.

```
[72]: a2

[72]: array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14])

[73]: a2=a2.reshape((3,5))

[74]: a2

[74]: array([[ 0,  1,  2,  3,  4],
             [ 5,  6,  7,  8,  9],
             [10, 11, 12, 13, 14]])
```

Línea 72

- Cuando se marca, la forma del objeto sigue siendo la misma. Por lo tanto, se debe asignar nuevamente para crear un nuevo objeto.

- La forma se puede cambiar directamente.

```
[75]: a3=np.arange(15)

[76]: a3

[76]: array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14])

[77]: a3.shape=(5,3)

[78]: a3

[78]: array([[ 0,  1,  2],
             [ 3,  4,  5],
             [ 6,  7,  8],
             [ 9, 10, 11],
             [12, 13, 14]])
```

Números aleatorios

- Los datos que siguen la distribución normal estándar (media 0, desviación estándar 1) se generan como números aleatorios. Consulte la ayuda (np.random.randn) para obtener más detalles.

```
[79]: data=np.random.randn(2,3)

[80]: data

[80]: array([[-0.76121445, -0.88421525, -1.95536064],
             [-0.30173803, -0.64876617, -0.33418645]])

[81]: np.mean(data)

[81]: -0.814246829732782
```

Línea 79

- Los datos generados siguen la distribución normal estándar con formas en dos filas y tres columnas.

- Creando data2 con 100 filas y 100 columnas

```
[82]: data2=np.random.randn(100,100)

[83]: np.mean(data2)

[83]: 0.010594191065786908

[84]: data2

[84]: array([[ 1.68446148,  0.91125632,  0.42086692, ..., -0.11175121,
                0.15063307,  0.2671715 ],
              [-1.59768225,  0.9348969 ,  0.67216623, ...,  2.07259064,
               -1.25988096,  2.08723447],
              [-1.07508385,  1.46934244, -0.94269049, ...,  1.00547245,
                0.64408887, -1.73139178],
              ...,
              [ 1.51995223, -0.74800658, -0.8759619 , ...,  0.64438961,
               -0.87140433,  0.75504121],
              [-1.4937868 ,  0.28359268, -0.45080111, ...,  0.21933063,
                1.22273758, -1.70514001],
              [-0.73627855, -0.65621757,  1.50394178, ..., -1.27475979,
                1.78418063,  0.71812189]])
```

- Continúe y cree data3

```
[85]: data3=np.random.randn(100,100)
      np.mean(data3)

[85]: 0.017739600155021105

[86]: data3

[86]: array([[-0.0516067 , -0.54704786, -1.85158139, ...,  0.11434929,
               0.93227992,  0.19873268],
             [ 0.09976707, -0.86177704,  1.23892015, ..., -0.75841696,
              -0.34717415, -0.04472912],
             [ 0.66190119,  0.44969343,  1.38969272, ..., -1.93715981,
               0.51056389,  1.98311285],
             ...,
             [-0.04475455, -0.76428618,  0.06937734, ..., -1.19997758,
              -1.20844941,  0.58158585],
             [-0.05328338, -0.59166801, -0.9036079 , ...,  0.01307157,
               0.030166  ,  0.87815587],
             [ 0.94328902, -0.69814295,  1.31254272, ...,  2.14712351,
              -0.08797109, -0.33515904]])
```

- El promedio es cercano a cero pero no completamente cero. Si se generan más datos, los resultados se acercan a cero.

- Comparación de listas de Python con NumPy Arrays: velocidad de procesamiento (1/4)
  - Comparemos una lista de Python y un NumPy array que almacena un millón de enteros.
  - Si ejecuta el siguiente ejemplo, puede ver que el código que usa NumPy es mucho más rápido que el código escrito con Python puro.
  - Al acercarse a los objetos np, indicados con la abreviatura NumPy, puede crear 1 millón de Arrays a partir de 0 utilizando el método arange.

```
[87]: np.arange(1000000)

[87]: array([     0,      1,      2, ..., 999997, 999998, 999999])
```

  - Aquí hay una estructura de datos de la lista para fines de comparación.

```
[88]: list(range(1000000))

[88]: [0,
       1,
       2,
       3,
       4,
       5,
```

- Comparación de listas de Python con NumPy Arrays: velocidad de procesamiento (2/4)

```
[89]: type(np.arange(1000000))

[89]: numpy.ndarray
```

Línea 89

- Devuelve el resultado en forma de NumPy.ndarray. Esto significa que hay un array con n dimensiones.

- Comparación de listas de Python con NumPy Arrays: velocidad de procesamiento (3/4)
  - A continuación se muestra la velocidad del Python array y la operación de multiplicar cada valor de 1 millón de listas por 2, 1000 veces.

```
[90]: %time for i in range(1000): np.arange(1000000)*2

      CPU times: total: 2.14 s
      Wall time: 2.28 s
```

  - %time es un comando mágico de IPython que devuelve un tiempo de ejecución único. Este comando mágico es un comando especial diseñado para controlar fácilmente tareas generales y otras operaciones en el sistema IPython.
  - Los comandos mágicos están etiquetados con un signo %.

```
[91]: %time for i in range(1000): list(range(1000000))*2

      CPU times: total: 34.6 s
      Wall time: 35.8 s
```

  - Tardó 35,8 segundos. Hay una diferencia de más de 17 veces.
  - Una vez más, es importante comprender los pros y los contras del Arrays y las listas que aprendimos anteriormente para realizar operaciones de IA de Big Data a gran escala en el futuro. Esta sería una razón clara por la que debemos aprender y utilizar NumPy cuando operamos con este tipo de operaciones.

- Comparación de listas de Python con NumPy Arrays: velocidad de procesamiento (4/4)
  - Los resultados pueden diferir en el sistema de cada usuario, como acabamos de ver en la pantalla.
  - El tiempo de pared, también llamado tiempo de reloj de pared, es la suma del tiempo total que tarda el programa en ejecutarse y finalizar, incluida la CPU, E/S, subprograma, etc.
  - Para resumir la vectorización una vez más, la vectorización es una de las principales razones para usar NumPy.
  - La vectorización es la operación de colocar datos sin utilizar bucles ni indexación.
  - La vectorización de NumPy optimiza la operación de ubicación con código C compilado al realizarla fuera de la vista. Por ejemplo, el código de declaración de bucle para calcular el peso de Machine Learning utiliza la vectorización NumPy.

Agregar elementos a NumPy Arrays

- Se agrega un valor a los datos unidimensionales. El rango de la primera dimensión es 1. Consulte la ayuda (np.append) para obtener más detalles.

```
[92]: a=np.array([1,2,3])

[93]: b=np.append(a, [4,5,6])

[94]: b

[94]: array([1, 2, 3, 4, 5, 6])
```

- Método NumPy.append() (1/3)
  - Puede extender un NumPy array llamando al método NumPy.append(). Tenga en cuenta que, a diferencia de las listas, el operador más '+' no se puede utilizar para este propósito.
  - Cuando un array tiene orientaciones, puede especificar las direcciones. Para un 2D array, el argumento del eje establecido en 0 significa extensión vertical, mientras que 1 significa extensión horizontal.
  - NumPy.append() proporciona solo una vista. Para que los cambios permanezcan, el resultado debe asignarse a una variable.

- Método NumPy.append() (2/3)

```
[95]: a=np.array([[1,2],[3,4]])

[96]: a

[96]: array([[1, 2],
             [3, 4]])

[97]: b=np.append(a,[[9,9]], axis=0)

[98]: b

[98]: array([[1, 2],
             [3, 4],
             [9, 9]])
```

Línea 97

- eje = 0 es una fila.

- Método NumPy.append() (3/3)

```
[99]: c=np.append(a,[[9],[9]], axis=1)

[100]: c

[100]: array([[1, 2, 9],
              [3, 4, 9]])
```

Línea 99

- eje = 1 es una columna.

Eliminación de elementos de NumPy Arrays

- Método para eliminar elementos de Arrays (1/2)

```
[101]: a=np.array([[1,2,3],[4,5,6]])

[102]: a

[102]: array([[1, 2, 3],
              [4, 5, 6]])

[103]: np.delete(a, 0)

[103]: array([2, 3, 4, 5, 6])

[104]: a

[104]: array([[1, 2, 3],
              [4, 5, 6]])
```

Línea 103

- Elimine el elemento de índice correspondiente.

Línea 104

- Aunque el elemento fue eliminado, array sigue siendo el mismo porque no se realizó ninguna asignación.

- Método para eliminar elementos de Arrays (2/2)

```
[105]: np.delete(a, 0, axis=0)

[105]: array([[4, 5, 6]])

[106]: a

[106]: array([[1, 2, 3],
              [4, 5, 6]])

[107]: np.delete(a, 1, axis=1)

[107]: array([[1, 3],
              [4, 6]])
```

Línea 105

- Elimina la primera fila.

Línea 107

- Elimina la segunda columna.

Unidad 1.

Estructura de datos NumPy array para un rendimiento computacional óptimo

- 1.1. NumPy Arrays
- 1.2. Conceptos básicos de NumPy array
- 1.3. Operaciones numéricas array
- 1.4. Indexación y corte NumPy
- 1.5. array Transposición e intercambio de ejes

Operaciones básicas

- Anteriormente aprendimos que la característica única de la operación NumPy array es que los datos se pueden procesar colectivamente sin usar una declaración de bucle for, lo que se llama vectorización. En este caso, se aplican operaciones aritméticas entre Arrays del mismo tamaño en cada unidad de elemento del array.
  - Por esta razón, los populares Machine Learning y Deep Learning libraries, como scikit-learn, TensorFlow, PyTorch, etc., se crearon en base a NumPy.
  - Este es un resumen de las características de NumPy que hemos aprendido hasta ahora.
    - Un library de bajo nivel y alto rendimiento implementado en C
    - Admite operaciones ndarray array multidimensionales rápidas y con memoria efectiva
    - Admite varias funciones computacionales como álgebra lineal, generador de números aleatorios, transformada de Fourier, etc.

- Comparación de listas de Python y NumPy Arrays: operador +

```
[1]: import numpy as np

[2]: a=[1,2,3]

[3]: b=[4,5,6]

[4]: a+b

[4]: [1, 2, 3, 4, 5, 6]
```

Línea 4

- En una lista, el operador + significa conectar los elementos dados.

- Comparación de listas de Python y NumPy Arrays: operador +

```
[5]: arr1=np.array([1,2,3])
     arr2=np.array([4,5,6])

[6]: arr1+arr2

[6]: array([5, 7, 9])
```

Línea 6

- En un NumPy array, la suma se realiza por elementos.

- Si el shape del array es el mismo, puede realizar operaciones de suma, resta, multiplicación y división.

[7]:
```
print(arr1.ndim)
print(arr1.size)
print(arr1.shape)
```

```
1
3
(3,)
```

[8]:
```
print(arr2.ndim)
print(arr2.size)
print(arr2.shape)
```

```
1
3
(3,)
```

[9]:
```
print(arr1-arr2)
print(arr1*arr2)
print(arr1/arr2)
```

```
[-3 -3 -3]
[ 4 10 18]
[0.25 0.4  0.5 ]
```

- Comparación de listas python y matrices numpy: multiplicación

```
[10]: a=[1,2,3]
      3*a

[10]: [1, 2, 3, 1, 2, 3, 1, 2, 3]

[11]: b=np.array([1,2,3])
      3*b

[11]: array([3, 6, 9])
```

Línea 10

- En un list, el operador * significa repetición.

Línea 11

- La multiplicación en un numpy array devuelve el resultado de la multiplicación de cada elemento.

- repetir y tile

```
[12]: b

[12]: array([1, 2, 3])

[13]: np.repeat(b,3)

[13]: array([1, 1, 1, 2, 2, 2, 3, 3, 3])

[14]: np.tile(b,3)

[14]: array([1, 2, 3, 1, 2, 3, 1, 2, 3])
```

Línea 13

- En numpy, el método de repetición se utiliza para repetir cada elemento.

Línea 14

- En numpy, el método del mosaico se utiliza para repetir array.

- Cuando se utiliza el método del mosaico, el array se puede repetir en una forma bidimensional.

```
[15]: c=np.arange(9)

[16]: c=c.reshape(3,3)

[17]: c.shape

[17]: (3, 3)

[18]: c

[18]: array([[0, 1, 2],
             [3, 4, 5],
             [6, 7, 8]])

[19]: np.tile(c,3)

[19]: array([[0, 1, 2, 0, 1, 2, 0, 1, 2],
             [3, 4, 5, 3, 4, 5, 3, 4, 5],
             [6, 7, 8, 6, 7, 8, 6, 7, 8]])
```

- Las matrices también pueden realizar operaciones exponenciales.

```
[20]: arr2

[20]: array([4, 5, 6])

[21]: arr2**2

[21]: array([16, 25, 36])

[22]: arr1 / (arr2**2)

[22]: array([0.0625    , 0.08      , 0.08333333])
```

- El array también puede realizar operaciones de comparación. Después de comprobar si cada elemento coincide con las condiciones, devuelve Verdadero si coincide y Falso en caso contrario.

```
[23]: arr3=np.array([10,20,30,40])

[24]: arr3

[24]: array([10, 20, 30, 40])

[25]: arr3 > 10

[25]: array([False,  True,  True,  True])
```

- Funciones Universales (1/3)
  - Las funciones universales operan elemento por elemento en matrices completas. Consulte la ayuda (np.ufunc) para obtener más detalles.

```
[26]: x=np.array([0,1,2,3])

[27]: x**3

[27]: array([ 0,  1,  8, 27], dtype=int32)

[28]: np.sqrt(x)

[28]: array([0.        , 1.        , 1.41421356, 1.73205081])
```

- Funciones Universales (2/3)
  - Funciones numpy:

Función	Explicación	¿Función universal?
sin, cos, tan	Funciones trigonométricas	Sí
arcsin, arccos, arctan	Funciones trigonométricas inversas	Sí
round	round a un número determinado de decimales	Sí
floor	Devuelve el entero más pequeño más cercano	Sí
ceil	Devuelve el entero mayor más cercano	Sí
fix	Devuelve el entero más cercano a 0	Sí
prod	Devuelve el producto de los elementos array.	No
cumsum	Devuelve las sumas acumuladas de un array	No
sum, mean, var, std, median	Funciones estadísticas	No
exp, log	Funciones exponenciales y logarítmicas	Sí
unique	Devuelve los valores únicos de un array	No
min, max, argmax, argmin	Mínimo, máximo y los índices correspondientes	No

https://numpy.org/devdocs/reference/

- Funciones Universales (3/3)
  - Métodos estadísticos de matrices numpy:

Método	Explicación
mean	Promedio
var	Diferencia
std	Desviación estándar
sum	Total sum
cumsum	Las sumas acumuladas de un array
max, min	El máximo y el mínimo de un array
argmax, argmin	Índices del máximo y del mínimo.

https://numpy.org/devdocs/reference/

- Métodos estadísticos de matrices numpy:

```
[29]: np.arange(1,11)

[29]: array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])

[30]: x=np.arange(1,11)

[31]: x.sum()

[31]: 55

[32]: x.mean()

[32]: 5.5
```

Línea 31

- sum

Línea 32

- Promedio

- Métodos estadísticos de matrices numpy:

```
[33]: x_mean=x.mean()

[34]: x-x_mean

[34]: array([-4.5, -3.5, -2.5, -1.5, -0.5,  0.5,  1.5,  2.5,  3.5,  4.5])

[35]: (x-x_mean)**2

[35]: array([20.25, 12.25,  6.25,  2.25,  0.25,  0.25,  2.25,  6.25, 12.25,
             20.25])
```

Línea 33

- Promedio 5.5 5.5

Línea 34

- Desviación (diferencia del promedio)

Línea 35

- Cuadrado de desviación

- Métodos estadísticos de matrices numpy:

```
[36]: dev=(x-x_mean)**2

[37]: dev.sum()

[37]: 82.5
```

Línea 37

- sum de desviación al cuadrado

- Métodos estadísticos de matrices numpy:

```
[38]: dev.sum() / x.size

[38]: 8.25
```

Línea 38

- El sum de las desviaciones al cuadrado divididas por el número de elementos se llama varianza.

- Este resultado se puede calcular directamente mediante el proceso anterior, o se puede utilizar var().

```
[39]: x.var()

[39]: 8.25

[40]: np.sqrt(x.var())

[40]: 2.8722813232690143
```

Línea 40

- La raíz cuadrada de la varianza es la desviación estándar.

- Métodos estadísticos de matrices numpy:

```
[38]: dev.sum() / x.size

[38]: 8.25
```

Línea 38

- El sum de las desviaciones al cuadrado dividido por el número de matrices se llama varianza.

- Este resultado se puede calcular directamente mediante el proceso anterior o se puede utilizar el método std().

```
[41]: x.std()

[41]: 2.8722813232690143
```

- Métodos estadísticos de matrices numpy:

```
[42]: x.cumsum()

[42]: array([ 1,  3,  6, 10, 15, 21, 28, 36, 45, 55])
```

Línea 42

- El método cumsum() se utiliza para encontrar el sum acumulativo de x.

- Métodos estadísticos de matrices numpy:

```
[43]: x=np.arange(1,7)

[44]: x

[44]: array([1, 2, 3, 4, 5, 6])

[45]: x.reshape(2,3)

[45]: array([[1, 2, 3],
             [4, 5, 6]])

[46]: x.reshape(2,-1)

[46]: array([[1, 2, 3],
             [4, 5, 6]])

[47]: x.reshape(3,-1)

[47]: array([[1, 2],
             [3, 4],
             [5, 6]])
```

Línea 46

- De esta manera, puedes designarlo como dos filas y tres columnas. Sin embargo, si designa solo un fila, utilice -1 para inferir la dimensión restante.

- Métodos estadísticos de matrices numpy:
  - También se pueden utilizar funciones estadísticas para cada fila y columna.

```
[48]: x=x.reshape(2,3)

[49]: x

[49]: array([[1, 2, 3],
             [4, 5, 6]])

[50]: x.mean(axis=0)

[50]: array([2.5, 3.5, 4.5])

[51]: x.mean(axis=1)

[51]: array([2., 5.])
```

Línea 50

- El promedio de 1 y 4 es 2,5. Cuando el eje = 0, la operación se realiza a lo largo de las columnas.

Línea 51

- El promedio de 1, 2 y 3 es 2. Cuando el eje = 1, la operación se realiza a lo largo de las filas.

- Acerca de los números aleatorios en numpy

```
[52]: np.random.seed(123)
```

Línea 52

- Inicializar el valor inicial usando np.random generará un número aleatorio fijo al generar valores de números aleatorios.

- Acerca de los números aleatorios en numpy
  - Devuelve Enteros aleatorios de "bajo" (inclusive) a "alto" (exclusivo). Consulte la ayuda (np.random.randint).).

```
[53]: np.random.randint(10)

[53]: 2

[54]: np.random.randint(1,11)

[54]: 3
```

Línea 53

- Se devuelve un número aleatorio de enteros 0 a 9.

Línea 54

- Así es como devolverías un número aleatorio de enteros del 1 al 10.

- Acerca de los números aleatorios en numpy
  - Escriba el código a continuación para verificar si el código devolverá 10 cada vez.

[55]:
```
while True:
    num=np.random.randint(1,11)
    print(num)
    if (num==10):
        break
```

```
7
2
4
10
```

- Acerca de los números aleatorios en numpy

[56]:
```
while True:
    num=np.random.randint(1,11)
    print(num)
    if (num==11):
        break
```

```
7
2
1
2
10
1
```

Línea 56

- En este caso, no se devuelve 11, por lo que se repite indefinidamente. Haga clic en (interrumpir kernel) para detener la repetición.

- Resumen final
  - Funciones universales:

```
[57]: x=np.array([0, 1, 2, 3, 4, 5])

[58]: np.sqrt(x)

[58]: array([0.        , 1.        , 1.41421356, 1.73205081, 2.        ,
             2.23606798])

[59]: np.exp(x)

[59]: array([  1.        ,   2.71828183,   7.3890561 ,  20.08553692,
              54.59815003, 148.4131591 ])
```

- Resumen final
  - Funciones numpy:

```
[61]: a
```

```
[61]: array([ 3,  3,  7,  2,  4, 10,  7,  2,  1,  2, 10,  1,  1, 10,  4,  5,  1,
              1,  5,  2,  8,  4,  3,  5,  8,  3,  5,  9,  1,  8, 10,  4,  5,  7,
              2,  6,  7,  3,  2,  9,  4,  6,  1,  3,  7,  3,  5,  5,  7,  4,  1,
              7,  5,  8,  7,  8,  2,  6,  8, 10,  3,  5,  9,  2,  3,  2,  2,  4,
              6, 10,  1,  9,  2,  7,  4,  4,  6, 10,  8, 10,  3,  4,  4,  4,  9,
              7, 10,  8,  7,  4, 10,  7,  7,  7,  2,  4,  5,  4,  2,  1,  6,  9,
              7,  9, 10,  2,  1,  4,  2,  4,  5,  8,  7,  2,  5,  4,  4,  8,  7,
              9,  7,  5,  5,  8,  1,  1, 10,  9,  9,  5,  9,  7,  2,  7,  9,  8,
             10,  2,  8,  2,  8, 10,  9,  8,  2,  4,  2,  9,  8,  6,  2,  3,  6,
              3,  3, 10,  4,  3,  7,  8, 10,  2,  4,  9,  4,  8, 10, 10,  4,  4,
              6,  7,  1,  9,  8,  8,  5,  5,  6,  1,  9, 10,  3,  6,  2,  6, 10,
```

```
[62]: a.size
```

```
[62]: 1000
```

- Resumen final
  - Funciones numpy:

```
[62]: a.size

[62]: 1000

[63]: np.unique(a)

[63]: array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])

[64]: np.median(a)

[64]: 5.0

[65]: np.sum(a)

[65]: 5485
```

Línea 63

- Devuelve sólo valores únicos que no se repiten.

- Resumen final
  - Funciones numpy:

```
[66]: np.round(np.var(a), 2)

[66]: 8.19

[67]: np.round(np.mean(a), 2)

[67]: 5.48

[68]: np.round(np.std(a), 2)

[68]: 2.86
```

Línea 66

- Redondeado a dos decimales.

- Resumen final
  - Funciones numpy:

```
[69]: np.max(a)

[69]: 10

[70]: np.min(a)

[70]: 1
```

- Resumen final
  - Funciones numpy:

```
[71]: z = np.random.randint(10, size=1000)

[72]: np.unique(z)

[72]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

[73]: set(z)

[73]: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
```

- Resumen final
  - Funciones universales:

```
[74]: a = np.array([2, -3, 4, 5, 7, 0, 1, -1])

[75]: a.argmax()

[75]: 4

[76]: a.argmin()

[76]: 1
```

Línea 75

- Devuelve la ubicación del índice con el valor más alto en un número entero.

Línea 76

- Devuelve la ubicación del índice con el valor más bajo en un número entero.

- Lineal Álgebra del numpy array
  - numpy admite no solo operaciones simples de matrices sino también operaciones matriciales (bidimensionales array) para Álgebra lineal.
  - Entre los diversos métodos, veamos primero cómo obtener la multiplicación de matrices, la matriz transpose, la matriz inversa y Determinante.

- Operaciones matriciales
  - El siguiente es el método para obtener el producto matricial, la matriz transpose, la matriz inversa y Determinante para matrices A y B.
    - producto matricial
    - Matriz transpose
    - matriz inversa
    - Determinante

- Operaciones matriciales
  - Para operaciones matriciales, haga 2x2 matrices A and B de la siguiente manera:

```
[77]: A = np.array([0, 1, 2, 3]).reshape(2,2)
      A

[77]: array([[0, 1],
             [2, 3]])

[78]: B = np.array([3, 2, 0, 1]).reshape(2,2)
      B

[78]: array([[3, 2],
             [0, 1]])
```

- Operaciones matriciales
  - Ejemplo del producto de matrices A y B (A*B). Se pueden utilizar ambos métodos.

```
[79]: A.dot(B)

[79]: array([[0, 1],
             [6, 7]])

[80]: np.dot(A, B)

[80]: array([[0, 1],
             [6, 7]])
```

- Operaciones matriciales
  - El siguiente es un ejemplo de cómo encontrar la matriz transpose de la matriz A. Se pueden utilizar ambos métodos.

```
[81]: np.transpose(A)

[81]: array([[0, 2],
             [1, 3]])

[82]: A.transpose()

[82]: array([[0, 2],
             [1, 3]])
```

- Operaciones matriciales
  - Cómo encontrar la matriz inversa de la matriz A

```
[83]: np.linalg.inv(A)

[83]: array([[-1.5,  0.5],
             [ 1. ,  0. ]])
```

  - Un ejemplo de obtención de un Determinante de matriz A

```
[84]: np.linalg.det(A)

[84]: -2.0
```

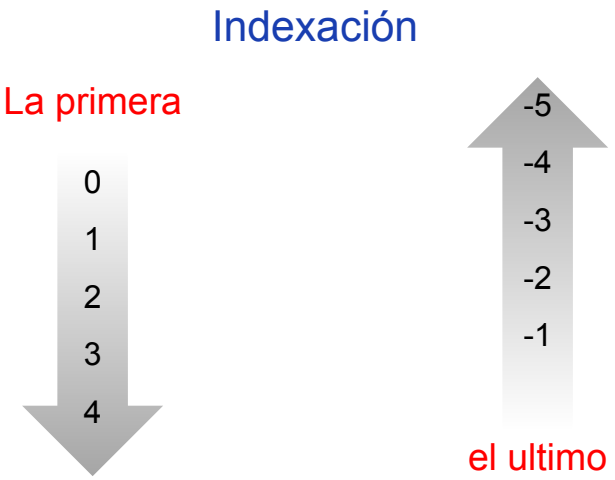
Unidad 1.

numpy array data: Estructura para un rendimiento computacional óptimo

- 1.1. Matrices numpy
- 1.2. numpy array Conceptos básicos
- 1.3. numpy array Operaciones
- 1.4. numpy Indexación y corte
- 1.5. array Transposición e intercambio de ejes

Indexación y corte

- Seleccionar elementos en un array especificando la ubicación o condición de un array se llama indexación. Y seleccionar elementos en un array especificando un rango se llama corte.
  - Para seleccionar un elemento en una ubicación específica en un array unidimensional, la posición del elemento debe especificarse como se muestra a continuación, y el elemento en array comienza en 0.
  - array Nombre[Ubicación]



- Indexación
  - Indexar un array unidimensional es como indexar python list.

```
[1]: import numpy as np

[2]: a=np.array([0,1,2,3])
     b=np.array([[0,1,2,3],
                 [4,5,6,7],
                 [8,9,10,11]])

[3]: a[0]

[3]: 0
```

- Indexación
  - También se pueden abordar matrices multidimensionales como la indexación list.

```
[4]: b[0]

[4]: array([0, 1, 2, 3])

[5]: b[0][1]

[5]: 1
```

Línea 5

- Puede acceder a un valor multidimensional específico.

- Indexación
  - A diferencia de la indexación list, las dimensiones se pueden dividir en comas (,) y abordarse en un array multidimensional. La dimensión dividida por comas se llama eje. Dado que b es array bidimensional, podemos tratarlo como una matriz. Se puede acceder a los valores de las columnas 1 y 2 como en la página anterior.
  - También se puede abordar como se muestra a continuación.

```
[6]: b[0,1]

[6]: 1
```

- Indexación
  - No solo se pueden importar los elementos del array, sino que el valor del índice correspondiente se puede cambiar de la siguiente manera.

```
[7]: a1=np.array([0,10,20,30,40,50])
     a1[5]

[7]: 50

[8]: a1[5]=70

[9]: a1

[9]: array([ 0, 10, 20, 30, 40, 70])
```

- Indexación
  - Para seleccionar múltiples elementos from y un array unidimensional, guárdelos de la siguiente manera.
  - Los corchetes exteriores son corchetes para indexación y los corchetes interiores son corchetes para list.
  - array nombre [[]]
  - array nombre [[posición 1, posición 2, ..., posición n]]

```
[10]: a1[[1,3,4]]

[10]: array([10, 30, 40])
```

Línea 10

- Los elementos 10, 30 y 40 ubicados en las posiciones 1, 3 y 4 se tomaron del unidimensional array a1.

- Indexación
  - Para seleccionar un elemento en una ubicación específica en un array bidimensional, especifique las posiciones de las filas y columnas de la siguiente manera.
  - array nombre [posición_fila, posición_columna],]
  - Si solo se ingresa el nombre array [posición_fila] sin una “posición_columna”, se selecciona todo el fila designado.
  - Es un método de selección e importación de un elemento específico mediante indexación en un array bidimensional.

```
[11]: a2=np.arange(10,100,10).reshape(3,3)
      a2

[11]: array([[10, 20, 30],
             [40, 50, 60],
             [70, 80, 90]])
```

- Indexación
  - En el array a2 bidimensional, un elemento que tiene una posición_fila de 0 y una posición_columna de 2 se selecciona e importa de la siguiente manera.

```
[12]: a2[0,2]

[12]: 30
```

  - Como se muestra a continuación, también puede cambiar el valor después de seleccionar un elemento especificando las posiciones de las filas y columnas en un array bidimensional.

```
[13]: a2[2,2]=95
      a2

[13]: array([[10, 20, 30],
             [40, 50, 60],
             [70, 80, 95]])
```

- Indexación
  - A continuación se muestra un método para obtener el fila completo especificando la "posición_fila" en un array bidimensional.

```
[14]: a2[1]

[14]: array([40, 50, 60])
```

  - El fila completo también se puede cambiar especificando un fila específico en un array bidimensional, como se muestra a continuación.

```
[15]: a2[2]=np.array([45,55,65])
      a2

[15]: array([[10, 20, 30],
             [40, 50, 60],
             [45, 55, 65]])
```

- Indexación
  - Para seleccionar varios elementos en un array bidimensional, haga lo siguiente.
  - array nombre [[posición_fila 1, posición_fila 2,…, posición_fila n],[posición_columna n], posición_columna 2,…, posición_columna n]] 1, 2, …, n],[n], 2, …, n]]
  - El siguiente es un ejemplo de selección de múltiples elementos especificando las posiciones de filas y columnas en un array bidimensional.

```
[16]: a2

[16]: array([[10, 20, 30],
             [40, 50, 60],
             [45, 55, 65]])

[17]: a2[[0,2],[0,1]]

[17]: array([10, 55])
```

Línea 17

- Si selecciona fila primero de [10, 20, 30] y columna de [45, 55, 65], se devuelven 10 y 55.

- Seleccionar el array especificando las condiciones
  - array Nombre[Condición]

```
[18]: a=np.array([1,2,3,4,5,6])

[19]: a[a>3]

[19]: array([4, 5, 6])

[20]: a[a%2==0]

[20]: array([2, 4, 6])
```

Línea 19

- Sólo se devuelven los elementos que cumplen las condiciones de “a>3”.

Línea 20

- Sólo se devuelven números pares.

Array Rebanar (slicing)

- En lugar de seleccionar un elemento mediante la indexación, el corte selecciona una parte de array especificando un range.

  - Para un array unidimensional, el corte especifica las posiciones del principio y el final, como se muestra a continuación.
  - array[posición_inicial:posición_final]]
  - Si no se especifica la posición inicial, la posición inicial se convierte en 0 y range se convierte en “from 0 a la posición final -1”. Si no se especifica la posición final, la longitud predeterminada es la array, por lo que range se convierte en "from start_position to the Fin de the array".

```
[21]: b1=np.array([0,10,20,30,40,50])
      b1[1:4]

[21]: array([10, 20, 30])
```

- rebanar
  - En un array unidimensional, el corte se realiza sin especificar una "posición_inicial" y una "posición_final", como se muestra a continuación.

```
[22]: b1[:3]

[22]: array([ 0, 10, 20])

[23]: b1[2:]

[23]: array([20, 30, 40, 50])
```

- rebanar
  - Ahora, consideremos el caso de un array bidimensional.

```
[24]: arr2d=np.arange(1,10).reshape(3,3)

[25]: arr2d

[25]: array([[1, 2, 3],
             [4, 5, 6],
             [7, 8, 9]])
```

  - Intente cortar el array de arriba.

```
[26]: arr2d[:2]

[26]: array([[1, 2, 3],
             [4, 5, 6]])
```

Línea 26

- Elija al comienzo de la segunda línea de "arr2d".

- rebanar
  - También es posible dividir matrices multidimensionales cruzando varios índices.

```
[27]: arr2d[:2, 1:]

[27]: array([[2, 3],
             [5, 6]])
```

  - Como se muestra arriba, el corte siempre ofrece una vista del array en la misma dimensión. Se pueden utilizar juntos un índice entero y un segmento para obtener un segmento de menor dimensión.
  - Por ejemplo, si desea seleccionar solo las dos primeras columnas del segundo fila, haga lo siguiente.

```
[28]: arr2d[1, :2]

[28]: array([4, 5])
```

- rebanar
  - Si selecciona solo el tercer columna en las dos primeras filas, haga lo siguiente.

```
[29]: arr2d[:2,2]

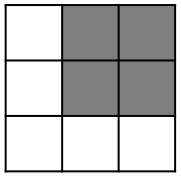
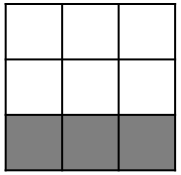
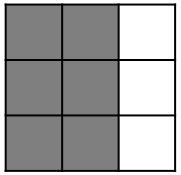
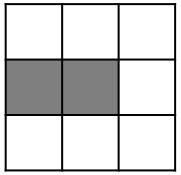
[29]: array([3, 6])
```

  - Si solo usa dos puntos, elige el eje completo, por lo que se devuelve la porción de la dimensión original.

```
[30]: arr2d[:, :1]

[30]: array([[1],
             [4],
             [7]])
```

- rebanar

	Código	Forma de rebanada
	arr[:2, 1:] 1:]	(2,2)
	arr[2] arr[2, :] :] arr[2:, :]	(3,) (3,) (1,3)
	arr[:, :2]	(3,2)
	arr[1, :2] arr[1:2, :2]	(2,) (1,2)

- Seleccionando en valores boolean

```
[31]: names=np.array(['Bob', 'Joe', 'Tom', 'Bob', 'Tom', 'Joe', 'Joe'])

[32]: names

[32]: array(['Bob', 'Joe', 'Tom', 'Bob', 'Tom', 'Joe', 'Joe'], dtype='<U3')
```

- Este boolean array determina si el nombre "Bob" está en los nombres array.

```
[33]: names=='Bob'

[33]: array([ True, False, False,  True, False, False, False])
```

- Al indexar bajo esta condición, solo se pueden devolver aquellos cuyo nombre sea "Bob".

```
[34]: names[names=='Bob']

[34]: array(['Bob', 'Bob'], dtype='<U3')
```

Indexación elegante

- Devuelve un nuevo array de shape y type dados sin inicializar las entradas. Consulte ayuda (np.empty) para obtener más detalles.

```
[35]: arr=np.empty((8,4))

[36]: for i in range(8):
          arr[i]=i

[37]: arr

[37]: array([[0., 0., 0., 0.],
             [1., 1., 1., 1.],
             [2., 2., 2., 2.],
             [3., 3., 3., 3.],
             [4., 4., 4., 4.],
             [5., 5., 5., 5.],
             [6., 6., 6., 6.],
             [7., 7., 7., 7.]])
```

- Indexación elegante
  - Al seleccionar filas en un orden específico, se puede omitir el ndarrray o el list que contiene el orden deseado.

```
[38]: arr[[4,3,0,6]]

[38]: array([[4., 4., 4., 4.],
             [3., 3., 3., 3.],
             [0., 0., 0., 0.],
             [6., 6., 6., 6.]])
```

  - Si utiliza números negativos como índice, seleccione un fila de al final.

```
[39]: arr[[-3,-5]]

[39]: array([[5., 5., 5., 5.],
             [3., 3., 3., 3.]])
```

- Indexación elegante
  - Para indexar valores que son solo múltiplos de 5, el boolean array de la condición se puede colocar en la variable y obtener.

```
[40]: arr2=np.arange(20)

[41]: arrMask=(arr2%5==0)

[42]: arr2[arrMask]

[42]: array([ 0,  5, 10, 15])
```

- Indexación elegante
  - También se puede obtener combinando dos condiciones.

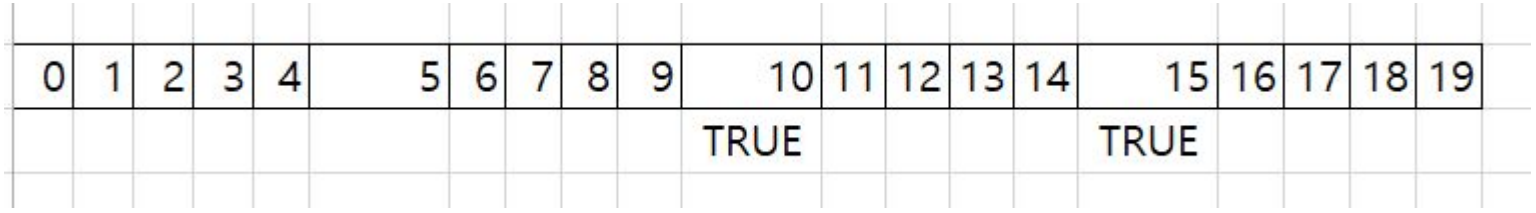
```
[43]: arr2>10

[43]: array([False, False, False, False, False, False, False, False, False,
             False, False,  True,  True,  True,  True,  True,  True,  True,
              True,  True])

[44]: arrMask2=(arr2>=10)

[45]: arr2[arrMask & arrMask2]

[45]: array([10, 15])
```



Unidad 1.

numpy array data: Estructura para un rendimiento computacional óptimo

- 1.1. Matrices numpy
- 1.2. numpy array Conceptos básicos
- 1.3. numpy array Operaciones
- 1.4. numpy Indexación y corte
- 1.5. array Transposición e intercambio de ejes

array Transposición

- array La transposición es una función especial que devuelve una vista en la que data shape ha cambiado sin copiar data. ndarray tiene un método transpose y una propiedad especial denominada T.

```
[1]: import numpy as np

[2]: arr=np.arange(15).reshape((3,5))
```

- array Transposición
  - transpose puede cambiar el orden dimensional del ndarray. T se puede utilizar para invertir el orden de todas las dimensiones con transpose. Dado que se trata de una función de uso frecuente, se convirtió en una función de acceso directo.

```
[3]: arr

[3]: array([[ 0,  1,  2,  3,  4],
            [ 5,  6,  7,  8,  9],
            [10, 11, 12, 13, 14]])

[4]: arr.transpose()

[4]: array([[ 0,  5, 10],
            [ 1,  6, 11],
            [ 2,  7, 12],
            [ 3,  8, 13],
            [ 4,  9, 14]])

[5]: arr

[5]: array([[ 0,  1,  2,  3,  4],
            [ 5,  6,  7,  8,  9],
            [10, 11, 12, 13, 14]])

[6]: arr.T

[6]: array([[ 0,  5, 10],
            [ 1,  6, 11],
            [ 2,  7, 12],
            [ 3,  8, 13],
            [ 4,  9, 14]])
```

- array Transposición
  - El Álgebra lineal, como la multiplicación y división de matrices, Determinante y la matriz cuadrada, es una parte importante de la biblioteca que trata con matrices.
  - Multiplicar dos matrices bidimensionales por operadores * produce el producto de cada elemento correspondiente, no la multiplicación de la matriz.
  - La multiplicación de matrices se calcula utilizando una función dot en el espacio de nombres numpy y un método array.

```
[7]: arr=np.random.randn(6,3)

[8]: arr

[8]: array([[ 0.06624326, -0.12347098,  1.54655281],
            [-0.02066961, -0.19496217,  1.58275183],
            [-0.29528588, -0.2906436 , -0.1858975 ],
            [-0.17043896, -0.72851337,  0.47749209],
            [-0.20749056,  0.04387839,  0.95461973],
            [ 0.5743246 , -0.93536745,  0.21142789]])

[9]: arr.T

[9]: array([[ 0.06624326, -0.02066961, -0.29528588, -0.17043896, -0.20749056,
              0.5743246 ],
            [-0.12347098, -0.19496217, -0.2906436 , -0.72851337,  0.04387839,
             -0.93536745],
            [ 1.54655281,  1.58275183, -0.1858975 ,  0.47749209,  0.95461973,
              0.21142789]])
```

- array Transposición
  - A menudo se usa en el cálculo matricial y np.dot también se usa para encontrar la parte interna de la matriz.

```
[10]: np.dot(arr.T, arr)

[10]: array([[ 0.49395967, -0.34046821, -0.03340285],
             [-0.34046821,  1.54529835, -0.94923617],
             [-0.03340285, -0.94923617,  6.11548612]])
```

- array Transposición
  - Primero, hagamos reshape un 2D array from de 3 filas y 4 columnas a 4 filas y 3 columnas.

```
[11]: arr_1=np.arange(12).reshape(3,4)

[12]: arr_1

[12]: array([[ 0,  1,  2,  3],
             [ 4,  5,  6,  7],
             [ 8,  9, 10, 11]])

[13]: arr_1.reshape((4,3))

[13]: array([[ 0,  1,  2],
             [ 3,  4,  5],
             [ 6,  7,  8],
             [ 9, 10, 11]])

[14]: arr_1.transpose((0,1))

[14]: array([[ 0,  1,  2,  3],
             [ 4,  5,  6,  7],
             [ 8,  9, 10, 11]])

[15]: arr_1.transpose((1,0))

[15]: array([[ 0,  4,  8],
             [ 1,  5,  9],
             [ 2,  6, 10],
             [ 3,  7, 11]])
```

- array Transposición
  - Esta vez, probemos con un array tridimensional.
  - Incluso en un array tridimensional, el método transpose recibe el número de eje de la tupla y lo reemplaza.

```
[16]: arr=np.arange(16).reshape((2,2,4))

[17]: arr

[17]: array([[[ 0,  1,  2,  3],
              [ 4,  5,  6,  7]],

             [[ 8,  9, 10, 11],
              [12, 13, 14, 15]]])
```

Línea 16

- Segunda página, segunda fila, cuarta fila.

- array Transposición

```
[18]: arr.transpose((1,0,2))

[18]: array([[[ 0,  1,  2,  3],
              [ 8,  9, 10, 11]],

             [[ 4,  5,  6,  7],
              [12, 13, 14, 15]]])

[19]: arr.transpose((0,1,2))

[19]: array([[[ 0,  1,  2,  3],
              [ 4,  5,  6,  7]],

             [[ 8,  9, 10, 11],
              [12, 13, 14, 15]]])

[20]: arr.transpose((2,1,0))

[20]: array([[[ 0,  8],
              [ 4, 12]],

             [[ 1,  9],
              [ 5, 13]],

             [[ 2, 10],
              [ 6, 14]],

             [[ 3, 11],
              [ 7, 15]]])
```

Línea 18

- Se invirtió el orden del primer y segundo eje y el último eje permaneció igual.

1.5. array Transposición e intercambio de ejes

- array Transposición
  - Hay un método llamado swapaxes en el ndarray, que recibe dos números de eje e invierte el array.

```
[21]: arr

[21]: array([[[ 0,  1,  2,  3],
              [ 4,  5,  6,  7]],

             [[ 8,  9, 10, 11],
              [12, 13, 14, 15]]])

[22]: help(arr.swapaxes)

Help on built-in function swapaxes:

swapaxes(...) method of numpy.ndarray instance
    a.swapaxes(axis1, axis2)

    Return a view of the array with `axis1` and `axis2` interchanged.

    Refer to `numpy.swapaxes` for full documentation.

    See Also
    --------
    numpy.swapaxes : equivalent function
```

- array Transposición

```
[23]: arr.swapaxes(1,2)
```

```
[23]: array([[[ 0,  4],
              [ 1,  5],
              [ 2,  6],
              [ 3,  7]],

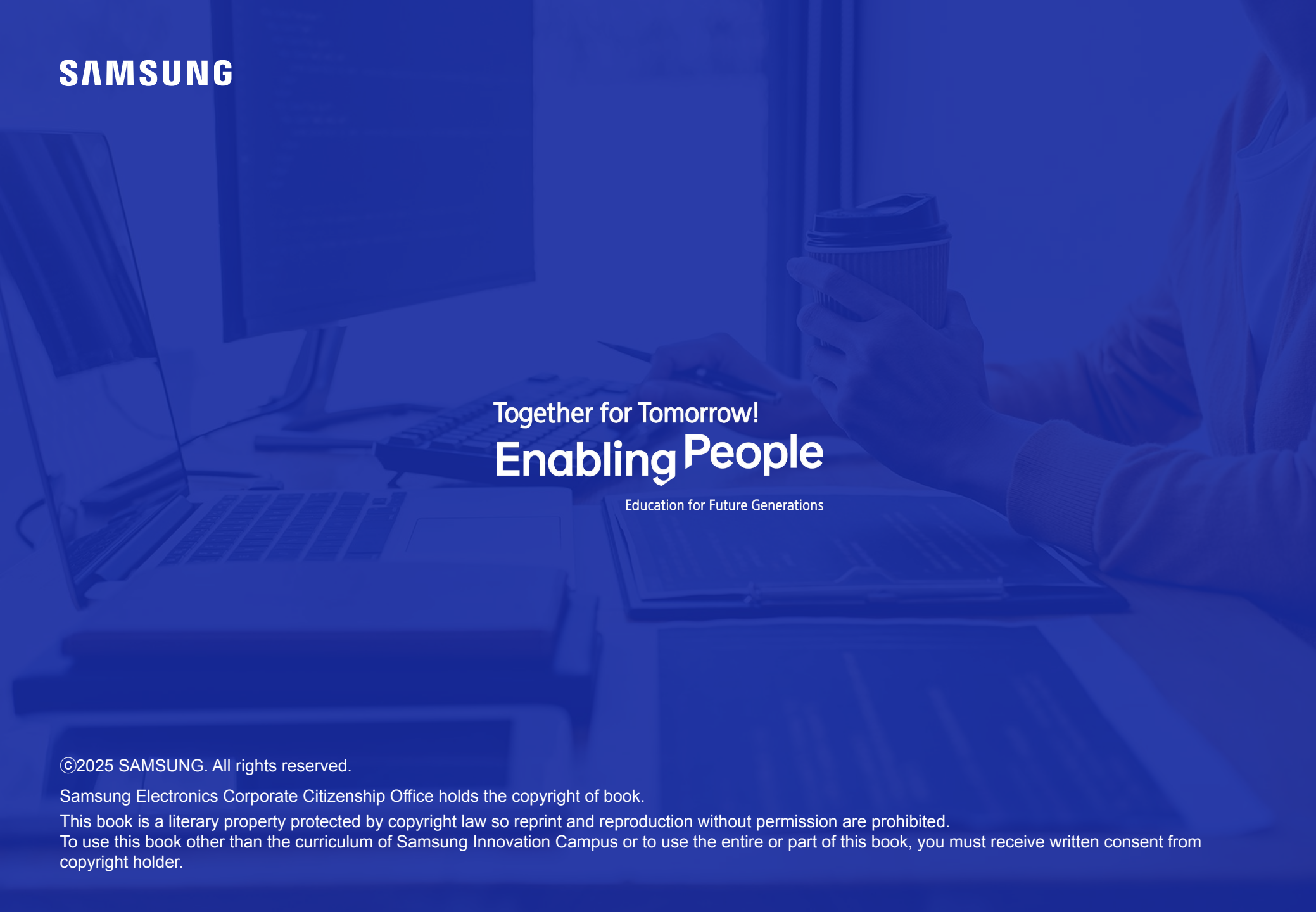
             [[ 8, 12],
              [ 9, 13],
              [10, 14],
              [11, 15]]])
```

```
[24]: arr.swapaxes(0,1)
```

```
[24]: array([[[ 0,  1,  2,  3],
              [ 8,  9, 10, 11]],

             [[ 4,  5,  6,  7],
              [12, 13, 14, 15]]])
```

SAMSUNG



ⓒ2025 SAMSUNG. All rights reserved.

Samsung Electronics Corporate Citizenship Office holds the copyright of book.

This book is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.
To use this book other than the curriculum of Samsung Innovation Campus or to use the entire or part of this book, you must receive written consent from copyright holder.